Intro to VHDL

Computer Architecture Course Season 3

Dr. AmirMahdi Hosseini Monazzah

Iran University of Science and Technology

1403-1404

1. From RTL to HDL

Register Transfer Level (RTL)

- Abstraction level describing digital systems
- Focuses on data flow between registers
- Describes operations performed on data

Hardware Description Language (HDL)

- VHDL and Verilog are the main HDLs
- Allows description of hardware at different abstraction levels
- Enables simulation and synthesis

Key Difference

RTL is a design methodology, HDL is the implementation language

1. RTL to VHDL Example

Register Transfer Level (RTL) Operation

$$EN : R1 \leftarrow A + B \quad \text{(at next clock edge)}$$

Element	**Meaning**
R1	Destination register
A, B	Source operands
$+$	Arithmetic operation
EN	Enable condition

Equivalent VHDL Implementation

```
process(clk)
begin
    if rising_edge(clk) then
        if enable = '1' then
            R1 <= A + B;
        end if;
    end if;
end process;
```

2. Basic VHDL Syntax I

Entity

Defines the interface (ports) of a component

```
1 entity FullAdder is
2     port (
3         a, b, cin : in  std_logic;
4         sum, cout : out std_logic
5     );
6 end FullAdder;
```

STD_LOGIC Type

STD_LOGIC type is equivalent of a wire.
It can have '0', '1', 'Z' (for disconnected / high-impedance) and some uncertain values.

2. Basic VHDL Syntax II

Architecture

Describes the implementation

```
1 architecture Behavioral of FullAdder is
2 begin
3     sum <= a xor b xor cin;
4     cout <= (a and b) or (cin and (a xor b));
5 end Behavioral;
```

First line almost means "place something to calculate XOR of a, b and cin. then connect it's output to sum output wire"
Instructions inside architechture are **not** evaluated sequentially.
Later updates on a, b or cin **will** cause update of sum, cout

Full Adder Example

```
1  entity FullAdder is
2      port (
3          a, b, cin : in  std_logic;
4          sum, cout : out std_logic
5      );
6  end FullAdder;
7
8  architecture Behavioral of FullAdder is
9  begin
10     sum <= a xor b xor cin;
11     cout <= (a and b) or (cin and (a xor b));
12 end Behavioral;
```

- a, b: Input bits
- cin: Carry in
- sum: Result
- cout: Carry out

3. Structural vs Behavioral Abstraction

Structural

Describes components and their connections

- Like a schematic
- Explicit instantiation
- Good for hierarchical design

Behavioral

Describes functionality

- Focus on what, not how
- More abstract
- Often more concise

4-to-1 Mux: Structural vs Behavioral

Structural

```
1  -- Structural 4-to-1 MUX
2  entity Mux4to1 is
3      port (
4          d0, d1, d2, d3 : in  std_logic;
5          sel            : in  std_logic_vector(1 downto 0);
6          y              : out std_logic
7      );
8  end Mux4to1;
9
10 architecture Structural of Mux4to1 is
11     signal not_sel0, not_sel1 : std_logic;
12     signal and0, and1, and2, and3 : std_logic;
13 begin
14     -- Invert select lines
15     not_sel0 <= not sel(0);
16     not_sel1 <= not sel(1);
17
18     -- AND gates for each data input
19     and0 <= d0 and not_sel1 and not_sel0;  -- sel = "00"
20     and1 <= d1 and not_sel1 and sel(0);    -- sel = "01"
21     and2 <= d2 and sel(1) and not_sel0;    -- sel = "10"
22     and3 <= d3 and sel(1) and sel(0);      -- sel = "11"
23
24     -- Final OR gate
25     y <= and0 or and1 or and2 or and3;
26 end Structural;
```

4-to-1 Mux: Structural vs Behavioral

Behavioral (when-else)

```
1  -- Behavioral 4-to-1 MUX
2  entity Mux4to1 is
3      port (
4          d0, d1, d2, d3 : in  std_logic;
5          sel            : in  std_logic_vector(1 downto 0);
6          y              : out std_logic
7      );
8  end Mux4to1;
9
10 architecture Behavioral of Mux4to1 is
11 begin
12     with sel select
13         y <= d0 when "00",
14              d1 when "01",
15              d2 when "10",
16              d3 when others;
17 end Behavioral;
```

`when-else` in VHDL is like

`condition ? value_true : value_false` in C

4. Signals, Vector

Signal

Signal is like an inner wire. Used to connect items inside an architecture and improve readability. Not an input nor an output.

STD_LOGIC_VECTOR

Represents a vector of STD_LOGICs, like a group of wires.

```
1 data_bus(3) <= '1';              -- Single bit assignment
2 lower_nibble <= data_bus(3 downto 0);  -- Slice assignment
3 data_bus <= (others => '0');     -- Aggregate assignment
```

There is also more types. e.g. representing signed or unsigned numbers using wires.

5. Component Instantiation and Port Map I

Component Declaration

```
1  architecture Behavioral of FA4 is
2      signal t: std_logic_vector(2 downto 0);
3      COMPONENT FA
4      PORT(
5           A : IN  std_logic;
6           B : IN  std_logic;
7           C_i : IN  std_logic;
8           S : OUT  std_logic;
9           C_o : OUT  std_logic
10         );
11     END COMPONENT;
12 begin
```

Define structure of another entity as a component to re-use it.

5. Component Instantiation and Port Map II

Component Instantiation

```
1     fa1: FA PORT MAP (
2          A => A(0),
3          B => B(0),
4          C_i => C_i,
5          S => S(0),
6          C_o => t(0)
7     );
```

Place an instance of FA component and connect wires using `PORT MAP`

5. Component Instantiation and Port Map III

Port Mapping

- Positional association

```
1 FA1: FullAdder port map(a, b, cin, sum, cout);
```

assignment is done by order of writing.

- Named association (recommended)

```
1 FA1: FullAdder
2     port map (
3         a => input1,
4         b => input2,
5         cin => carry_in,
6         sum => result,
7         cout => carry_out
8     );
```

6. Component Replication with for-generate

for-generate Statement

- Replicates hardware components or code sections
- Works like a loop: places multiple instances when simulation or synthesis

Basic Syntax

```
1    label_for: for i in 1 to 2 generate
2    fa_in_for: FA PORT MAP (
3        A => A(i),
4        B => B(i),
5        C_i => t(i - 1),
6        S => S(i),
7        C_o => t(i)
8    );
9    end generate;
```

Key Features

- Must use constants for bounds
- Supports nested generates
- Can include conditional generates
- Works with components and concurrent statements

Complete 4-bit Full Adder I

```
1  entity FA4 is
2      port(
3           A : IN  std_logic_vector(3 downto 0);
4           B : IN  std_logic_vector(3 downto 0);
5           C_i : IN  std_logic;
6           S : OUT  std_logic_vector(3 downto 0);
7           C_o : OUT  std_logic
8          );
9  end FA4;
10
11 architecture Behavioral of FA4 is
12     signal t: std_logic_vector(2 downto 0);
13     COMPONENT FA
14     PORT(
15          A : IN  std_logic;
16          B : IN  std_logic;
17          C_i : IN  std_logic;
18          S : OUT  std_logic;
19          C_o : OUT  std_logic
20         );
21     END COMPONENT;
22 begin
23     fa1: FA PORT MAP (
24         A => A(0),
25         B => B(0),
26         C_i => C_i,
27         S => S(0),
28         C_o => t(0)
29     );
30
31     label_for: for i in 1 to 2 generate
32     fa_in_for: FA PORT MAP (
```

Complete 4-bit Full Adder II

```
33          A => A(i),
34          B => B(i),
35          C_i => t(i - 1),
36          S => S(i),
37          C_o => t(i)
38      );
39      end generate;
40
41      fa_last: FA PORT MAP (
42          A => A(3),
43          B => B(3),
44          C_i => t(2),
45          S => S(3),
46          C_o => C_o
47      );
48  end Behavioral;
```

7. Process and Sequential Logic

Process Block

```
process(clk)
begin
    if rising_edge(clk) then
        q <= d;
    end if;
end process;
```

- Events like changing on sensitivity list (items in PARENTHESIS after `process` key word) triggers process execution
- Iinstructions inside process are executed **sequentially** (Like a normal C program). So things like placing component inside process is not allowed.

Shift Register Example I

```
1  entity ShiftReg4 is
2      port (
3          PI: IN STD_LOGIC_VECTOR(3 downto 0);
4          PO: OUT STD_LOGIC_VECTOR(3 downto 0);
5          CLK: IN STD_LOGIC;
6          SI: IN STD_LOGIC;
7          OP: IN STD_LOGIC_VECTOR(1 downto 0)
8      );
9  end ShiftReg4;
10
11 architecture Behavioral of ShiftReg4 is
12     signal value: STD_LOGIC_VECTOR(3 downto 0);
13 begin
14     PO <= value;
15     identifier : process (CLK)
16     begin
17         IF (rising_edge(CLK)) THEN
18             case(op) is
19                 when "01" =>
20                     value <= "0000";
```

Shift Register Example II

```
21                  when "10" =>
22                      value <= value(2 downto 0) & SI;
23                  when "11" =>
24                      value <= PI;
25                  when others =>
26                      value <= value;
27              end case ;
28          END IF;
29      end process ; -- identifier
30  end Behavioral;
```

- case-when in VHDL is like switch-case in C.
- & operator concatenates two vectors. here it makes a vector from four wires [value(2) value(1) value(0) SI].
- PO <= value is outside process. It's yet a connction from value to PO wire.

More on Sequential Logic

Note that code above does not explicitly have something like flip-flop or latch. Signals allow both reading and assignment, processes are executed sequentially, and using them together with clock-edge sensitivity (`rising_edge`/`falling_edge`) creates sequential logic that infers flip-flops during synthesis.